Lidar Guard

1. Course Content

- Run the sample program, which will detect the closest object to the vehicle in real-time and control the chassis to rotate and track it. At the same time, if the distance between the object and the car is less than the alarm threshold, the car's buzzer will sound.

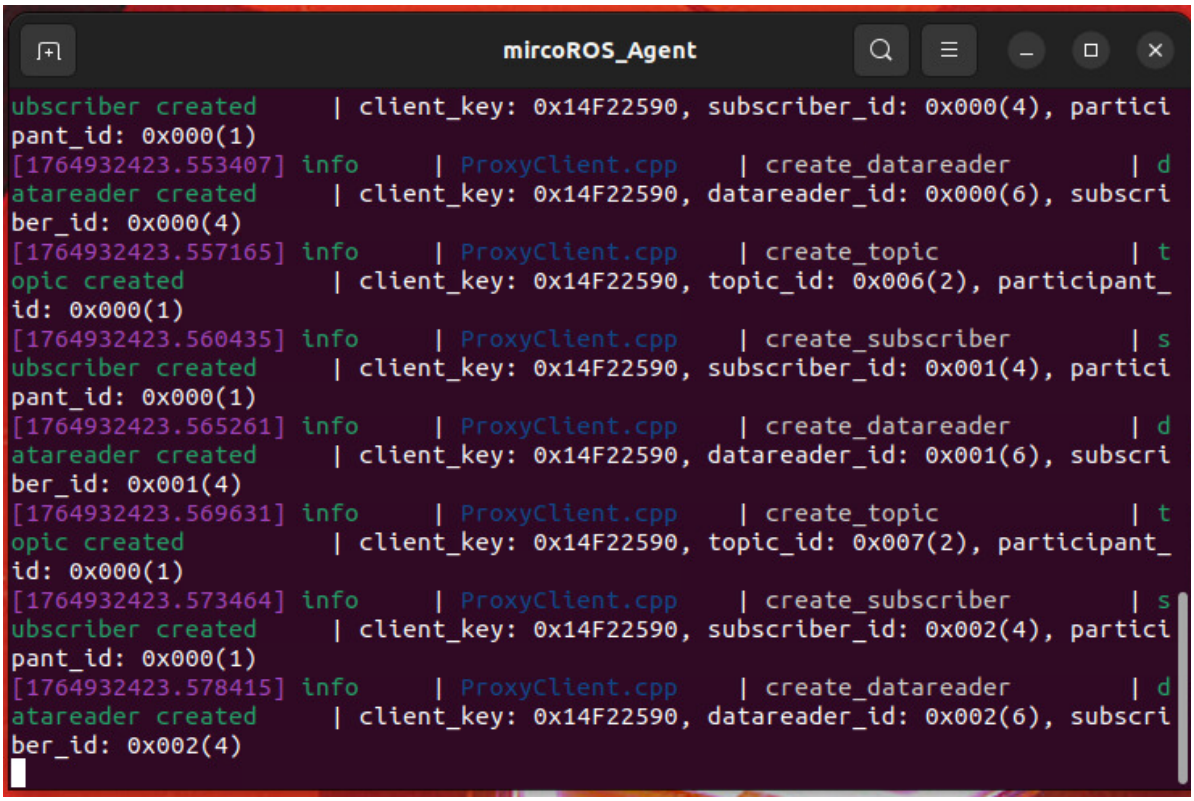
2. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



3. Run the Example

- Start the Lidar guard node

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Enter the command,

```
ros2 launch m3_bringup lidar_demo.launch.py demo:=laser_Warning
```

- After the program starts, the Lidar will scan for the closest object within its detection range. When the object is moved slowly from side to side, the program will control the chassis to rotate and track it, keeping the front of the car aligned with the object. If the distance between the car and the object is less than 0.55 meters, the car's buzzer will sound.

4. Core Code Analysis

- Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/lidar/laser_Warning.py
```

- Import necessary libraries:

```
#ros lib
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
from std_msgs.msg import UInt16
#commom lib
import math
import numpy as np
import time
from time import sleep
# Import libraries related to chassis PID control
from yahboom_M3Pro_laser.common import *
import os
```

- Program initialization and creation of publishers and subscribers:

```
def __init__(self,name):
    super().__init__(name)
    # Create a subscriber to the fused Lidar topic messages
    self.sub_laser =
self.create_subscription(LaserScan,"/scan",self.registerScan,1)
    # Create a subscriber to the joystick remote control topic messages
    self.sub_JoyState = self.create_subscription(Bool,'/JoyState',
self.JoyStateCallback,1)
```

```
    #create a pub
    # Create a publisher for the velocity topic messages
    self.pub_vel = self.create_publisher(Twist,'/cmd_vel',1)
    # Create a publisher for the buzzer topic messages
    self.pub_Buzzer = self.create_publisher(UInt16,'/beep',1)
    #declareparam
    self.declare_parameter("linear",0.5)
    self.linear =
self.get_parameter('linear').get_parameter_value().double_value
    self.declare_parameter("angular",1.0)
    self.angular =
self.get_parameter('angular').get_parameter_value().double_value
    # Lidar detection angle
    self.declare_parameter("LaserAngle",45.0)
    self.LaserAngle =
self.get_parameter('LaserAngle').get_parameter_value().double_value
    # Alarm distance, the buzzer will sound if the distance is less than this
value
    self.declare_parameter("ResponseDist",0.55)
    self.ResponseDist =
self.get_parameter('ResponseDist').get_parameter_value().double_value
    # Joystick control flag, if True, the car is controlled by the joystick.
Press R2 to take or enable joystick control.
    self.Joy_active = False
    # Create an angular velocity PID control object
    self.ang_pid = SinglePID(3.0, 0.0, 5.0)
```

`registerScan` Lidar topic callback function:

```
def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return
    ranges = np.array(scan_data.ranges)
    minDistList = []
    minDistIDList = []

    for i in range(len(ranges)):
        # Convert radians from Lidar data to degrees
        angle = (scan_data.angle_min + scan_data.angle_increment * i) * RAD2DEG
        # If the current angle is within the detection range and the distance is
not 0
        if abs(angle) < self.LaserAngle and ranges[i] !=0.0 :
            #print("angle: ",angle)
            # Add the distance at this angle to the minimum distance list
            minDistList.append(ranges[i])
            # Add this angle to the minimum ID list
            minDistIDList.append(angle)
    if len(minDistList) == 0: return
    # Find the minimum value in the distance list
    minDist = min(minDistList)
    # Find the angle corresponding to the shortest distance
    minDistID = minDistIDList[minDistList.index(minDist)]
    print("minDistID: ",minDistID)
    # If self.Joy_active is True, publish a stop velocity and exit the callback
    if self.Joy_active :
        self.pub_vel.publish(Twist())
        return
```

```
    print("minDist: ",minDist)
    # If the car's distance is less than the alarm distance
    if minDist <= self.ResponseDist and minDist!=0.0:
        b = UInt16()
        b.data = 1
        self.pub_Buzzer.publish(b)
    else:
        self.pub_Buzzer.publish(UInt16())
    velocity = Twist()
    print("minDistID: ",minDistID)
    # Calculate angular velocity
    ang_pid_compute = self.ang_pid.pid_compute(abs(minDistID)  / 72, 0)
    # If the minimum angle is greater than 0, it means the object is on the left,
so the car turns right
    if 0<minDistID :
        velocity.angular.z = ang_pid_compute
    # If the minimum angle is less than 0, it means the object is on the right,
so the car turns left
    elif minDistID <0:
        velocity.angular.z = -ang_pid_compute
    print("orin_angular.z: ",velocity.angular.z)
    if abs(ang_pid_compute) < 0.5: velocity.angular.z = 0.0
    velocity.angular.z = velocity.angular.z *0.5
    print("angular.z: ",velocity.angular.z)
    # Publish the velocity control topic
    self.pub_vel.publish(velocity)
```